

后端架构重构与平台实现汇报

Backend Architecture Refactoring & Platform Implementation Report



报告人：xxx

时 间：2026 年 xx 月 xx 日

CONTENTS 目录



01

实现范围

本次交付内容与实现结果

02

后端架构

gateway、public-service、QA 服务协作结构

03

核心实现

管理后台、任务链路、消息存储、MinIO 原文

04

交付结果

前端配套、部署、当前状态与后续完善

本次实现范围与交付结果

围绕后端重构、管理后台、任务模型、消息存储和部署交付，完成系统级实现。

已实现：后端服务结构

完成 gateway 统一入口与路由编排，
拆分 public-service 承接公共接口，
保留 fastQA / highThinkingQA / patent 执行边界，
前端调用和后端服务拓扑实现解耦。

已实现：管理后台

完成登录鉴权、安全问题、用户管理，
实现部门字典、人员主档、账号人员绑定，
支持管理员维护与用户侧信息补全，
组织人员数据形成可追踪闭环。

已实现：任务执行模型

实现 task_id 生命周期和任务状态维护，
支持页面刷新后重新附着真实任务，
支持多任务并行、任务取消和事件回放，
长任务执行不依赖单次前端页面状态。

已实现：存储与交付

实现消息持久化和 assistant 终态落库，
实现 MinIO 文件上传与原文对象读取，
Redis 承接任务状态、队列和短期缓存，
补充前端配套页面与部署检查链路。

交付结果：实现统一入口、公共服务拆分、可恢复任务模型、消息落库、文件与原文对象读取。

实现结果一

服务入口统一到 gateway，
public-service 承接公共状态和接口，
QA 服务按模式保持独立执行。

实现结果二

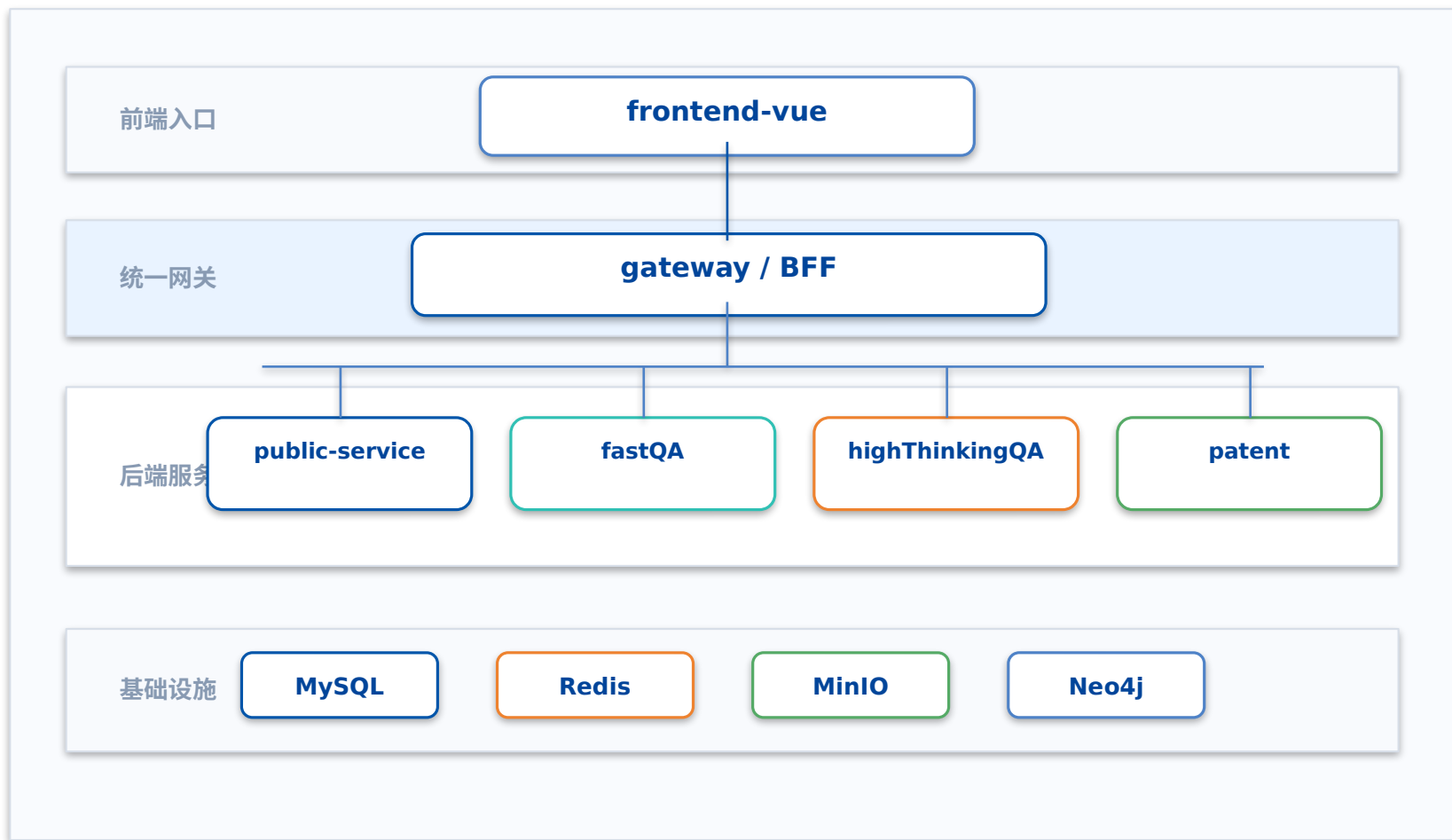
管理员、组织、人员、绑定、认证
均完成后端接口和前端页面配套，
管理数据可集中维护。

实现结果三

任务、消息、文件和原文读取链路
已经贯通到 Redis / MySQL / MinIO，
刷新、取消、回放都有后端状态依据。

实现后的后端服务结构

前端统一进入 gateway，公共服务和多模式 QA 服务按职责协作。



已实现：统一入口

gateway 承接前端请求，实现鉴权透传、路由分发和 SSE 转发。前端调用保持统一，服务调整不影响页面入口。

已实现：公共服务

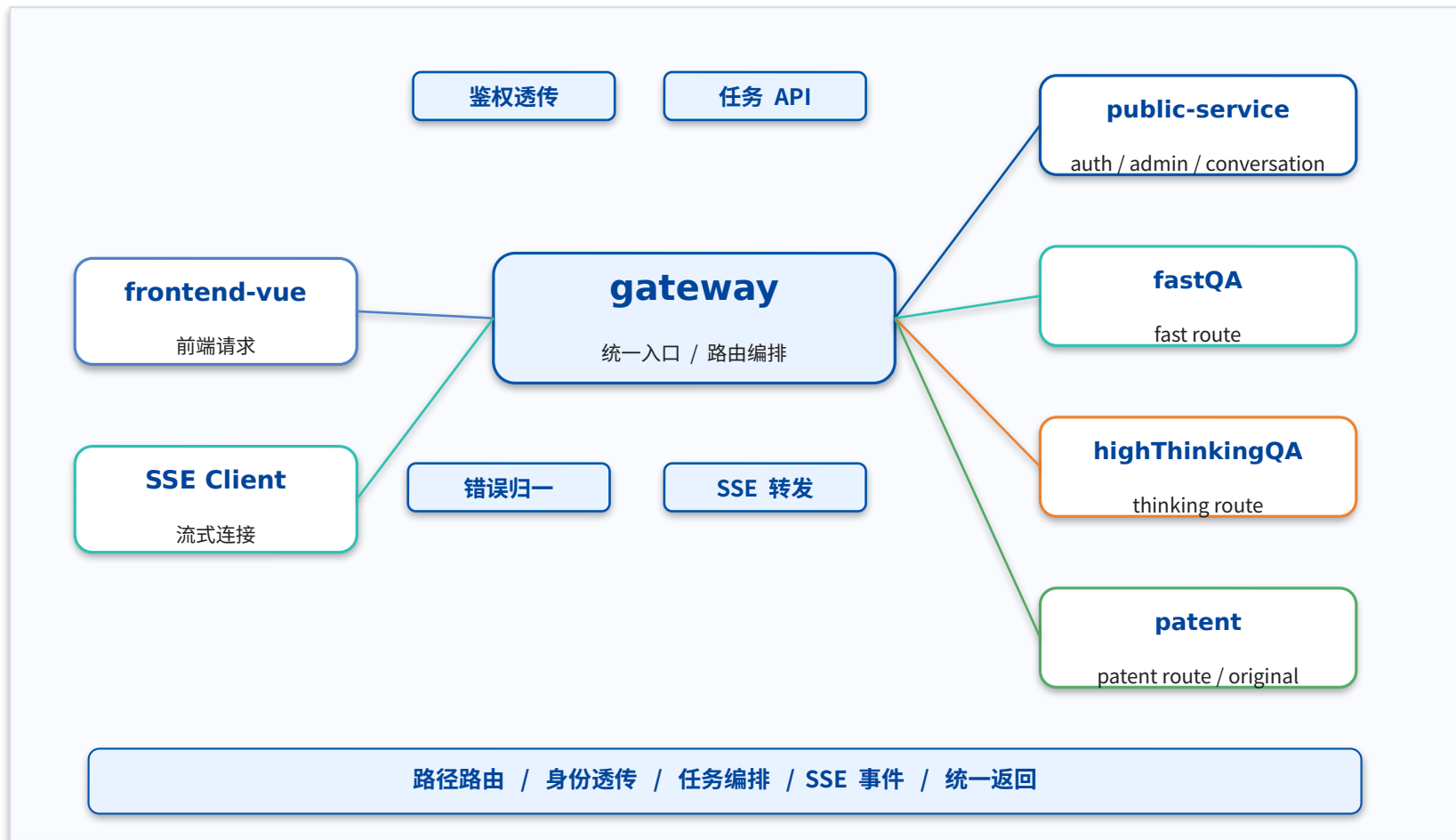
public-service 接管认证、会话、文件、组织管理和系统接口。用户、会话、文件等状态统一收敛。

已实现：执行边界

fastQA、highThinkingQA、patent 按问答模式独立执行。MySQL、Redis、MinIO 为任务和对象提供落点。

实现 gateway 统一入口与服务路由

前端请求统一进入 gateway，再按公共接口、任务 API 和 QA 模式分流。



已实现：前端统一调用

前端请求统一进入 gateway，
页面侧不直接感知 public-service、QA 服务地址。
鉴权、会话、任务、原文接口按路径分发。

已实现：请求分流

公共接口代理到 public-service，
QA 请求按 fast / thinking / patent 模式进入执行服务。
任务 API 在入口侧形成统一调度面。

已实现：SSE 承接

SSE 事件由 gateway 统一转发，
事件序号与 task_id 一起用于刷新恢复。
页面刷新后仍可回到真实任务进度。

实现 public-service 公共服务拆分

认证、会话、文档、组织和系统接口由 public-service 统一承接。

auth

登录 / 注册
安全问题 / 用户状态

conversation

会话 / 消息
任务绑定 / 历史回放

uploads

文件上传
file_id / storage_ref

documents

原文读取 / 预览
文档接口 / 对象引用

admin

用户 / 部门 / 人员
绑定关系 / 批量导入

quota & system

配额接口
健康检查 / 系统接口

实现后：公共状态进入 public-service，入口编排进入 gateway，QA 服务聚焦执行链路。

已实现：认证会话

登录、注册、安全问题与会话消息
统一由 public-service 承接。
用户消息、占位消息和终态消息有统一记录。

已实现：组织管理

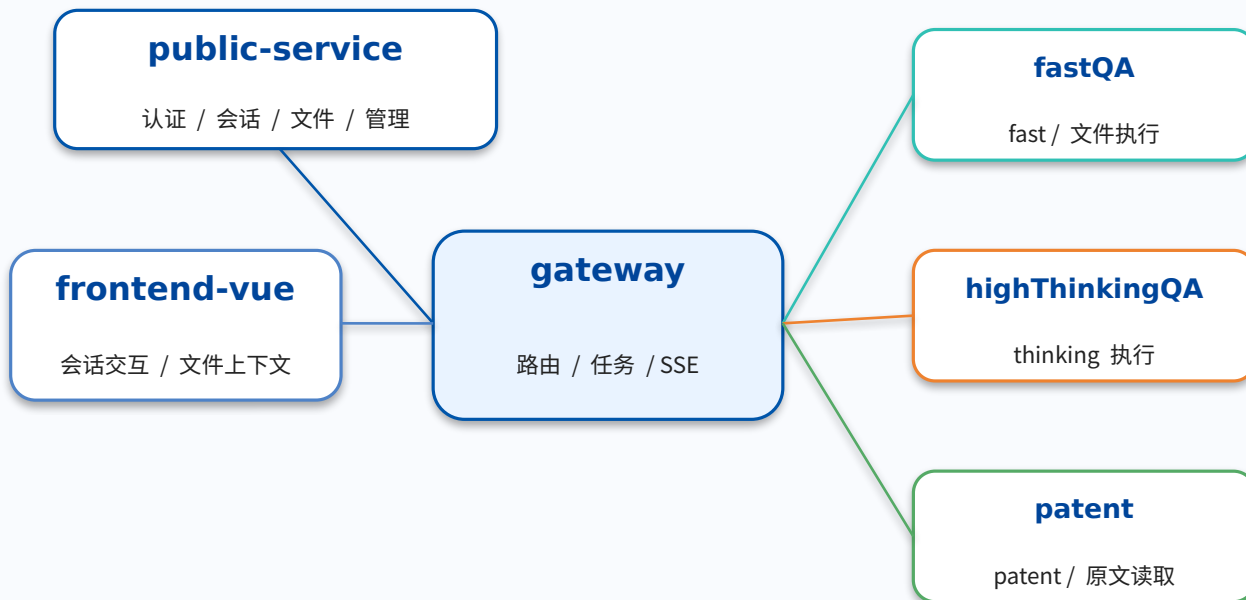
用户、部门、人员和绑定关系集中维护，
管理员页面直接调用相关接口。
人员信息补全与账号绑定形成闭环。

已实现：系统接口

上传、文档、健康检查和配额接口
从 QA 服务中拆出统一承接。
部署检查和运行维护都有公共入口。

实现多模式 QA 执行服务边界

不同问答模式保留独立执行链路，公共接口通过 gateway / public-service 协作。



实现后：公共接口统一协作，fastQA / highThinkingQA / patent 保留各自执行链路。

已实现：执行分离

fastQA、highThinkingQA、patent
保留各自执行链路与上下文处理。
gateway 只做入口编排、任务控制和转发。

已实现：状态协作

会话、文件、权限与任务状态
通过 gateway / public-service 协同。
执行服务按 file_id、task_id 等引用获取上下文。

已实现：原文读取

patent 原文查看接入 MinIO 对象链路，
答案中的原文引用可跳转到存储对象。
原文读取不混入其他 QA 模式执行链路。

实现管理员后台与组织人员管理

账号、部门、人员、绑定关系完成后台维护和用户侧补全闭环。

已实现：账号管理

登录鉴权
用户停启用
密码重置
安全问题
用户状态控制

已实现：部门管理

三级部门字典
部门停启用
批量导入
字典查询
组织结构维护

已实现：人员主档

人员主档维护
工号姓名管理
校验码校验
绑定校验
信息补全

已实现：绑定关系

账号绑定人员
管理员修复绑定
用户自助绑定
个人中心补全
关系可追踪

管理员侧：用户、部门、人员、绑定集中维护

用户侧：注册、个人中心、人员信息自助补全

已实现：后台维护

管理员页面覆盖用户、部门、人员、绑定关系和批量导入。
组织基础数据可以在后台统一维护。

已实现：主档校验

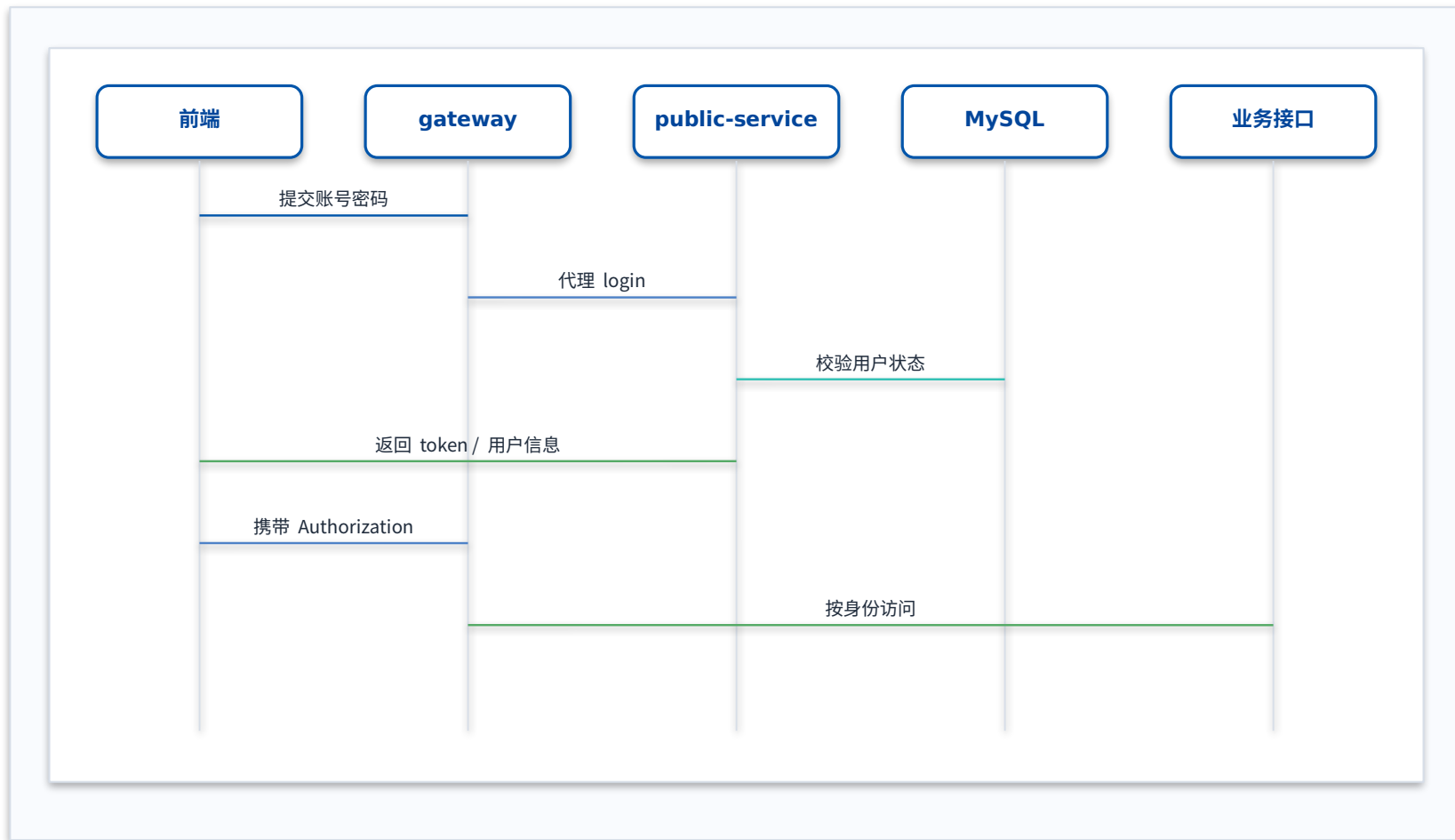
三级部门字典和人员主档参与校验，
工号、姓名、校验码用于绑定确认。
注册和个人中心可复用同一套数据。

已实现：绑定闭环

管理员可修复绑定关系，用户可自助绑定。
账号、人员、部门关系可追踪。
用户状态与人员信息保持一致。

实现登录鉴权与用户状态链路

登录、鉴权、安全问题和用户状态校验统一进入 public-service。



已实现：认证入口

登录请求经 gateway 代理到 public-service，前端获取统一 token 与用户信息。注册、安全问题、找回相关接口使用同一认证域。

已实现：状态校验

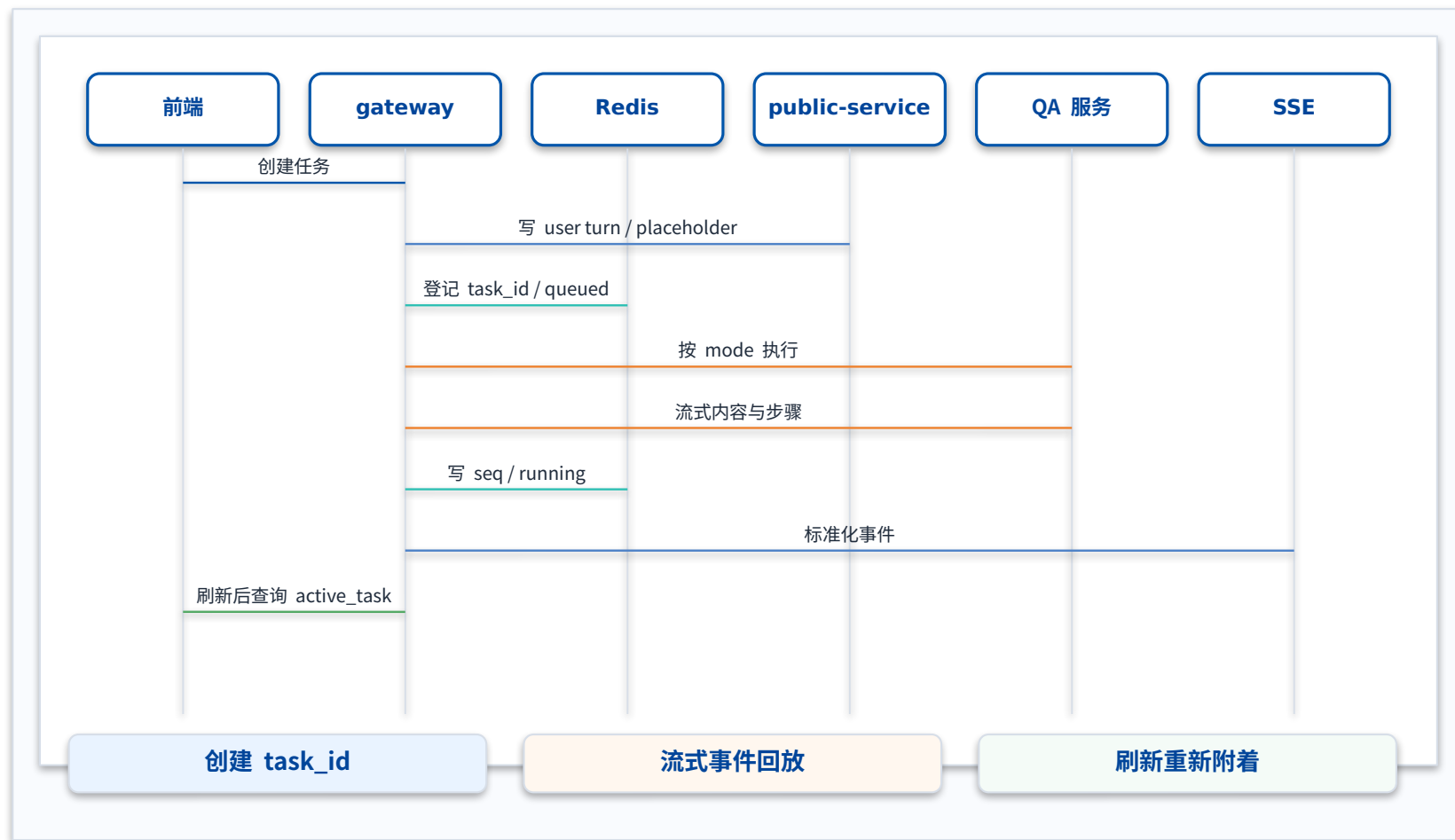
认证阶段读取用户状态和绑定信息，停启用、角色等控制在入口生效。管理员侧调整后可影响后续访问。

已实现：身份透传

后续接口携带 Authorization，由 gateway 透传到对应业务服务。公共接口和 QA 接口保持同一身份上下文。

实现可恢复的问答任务执行链路

创建、流式返回和刷新恢复统一围绕 task_id 推进。



已实现：任务标识

创建任务时生成并登记 task_id，
queued、running、completed 等状态可追踪。
流式事件围绕同一任务标识持续推进。

已实现：刷新恢复

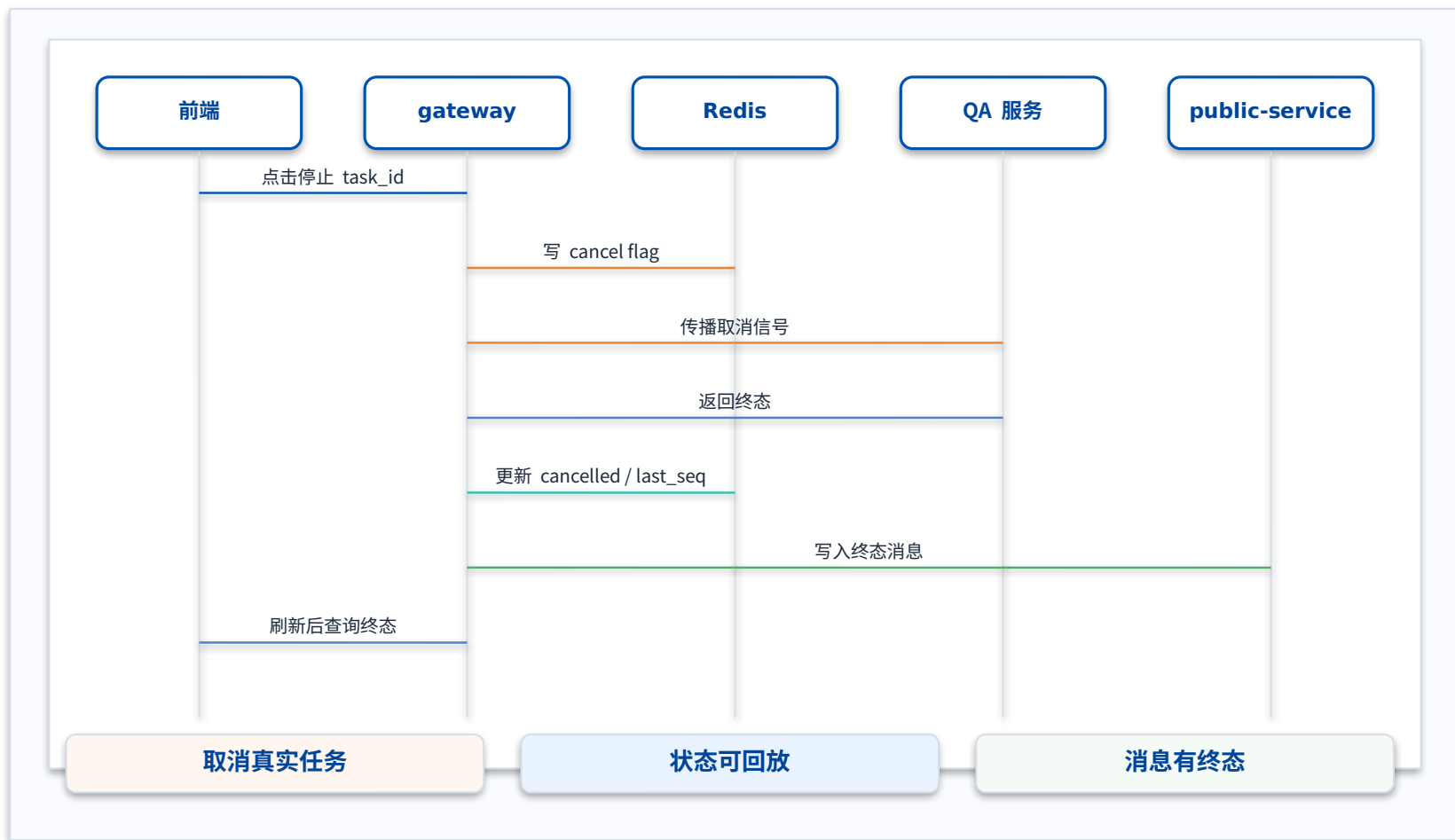
页面刷新后查询 active_task，
再根据事件序号重新附着到任务。
已产生的流式内容可以按序补齐。

已实现：并行任务

任务状态写入 Redis，
多个问答任务可独立推进。
不同会话任务互不覆盖，前端可分别恢复。

实现任务取消、事件回放与消息终态

取消、事件回放和终态落库统一收敛到任务状态与会话消息。



已实现：任务取消

停止请求写入 cancel flag ,
并向执行服务传播取消信号。
任务停止与前端按钮状态保持一致。

已实现：事件回放

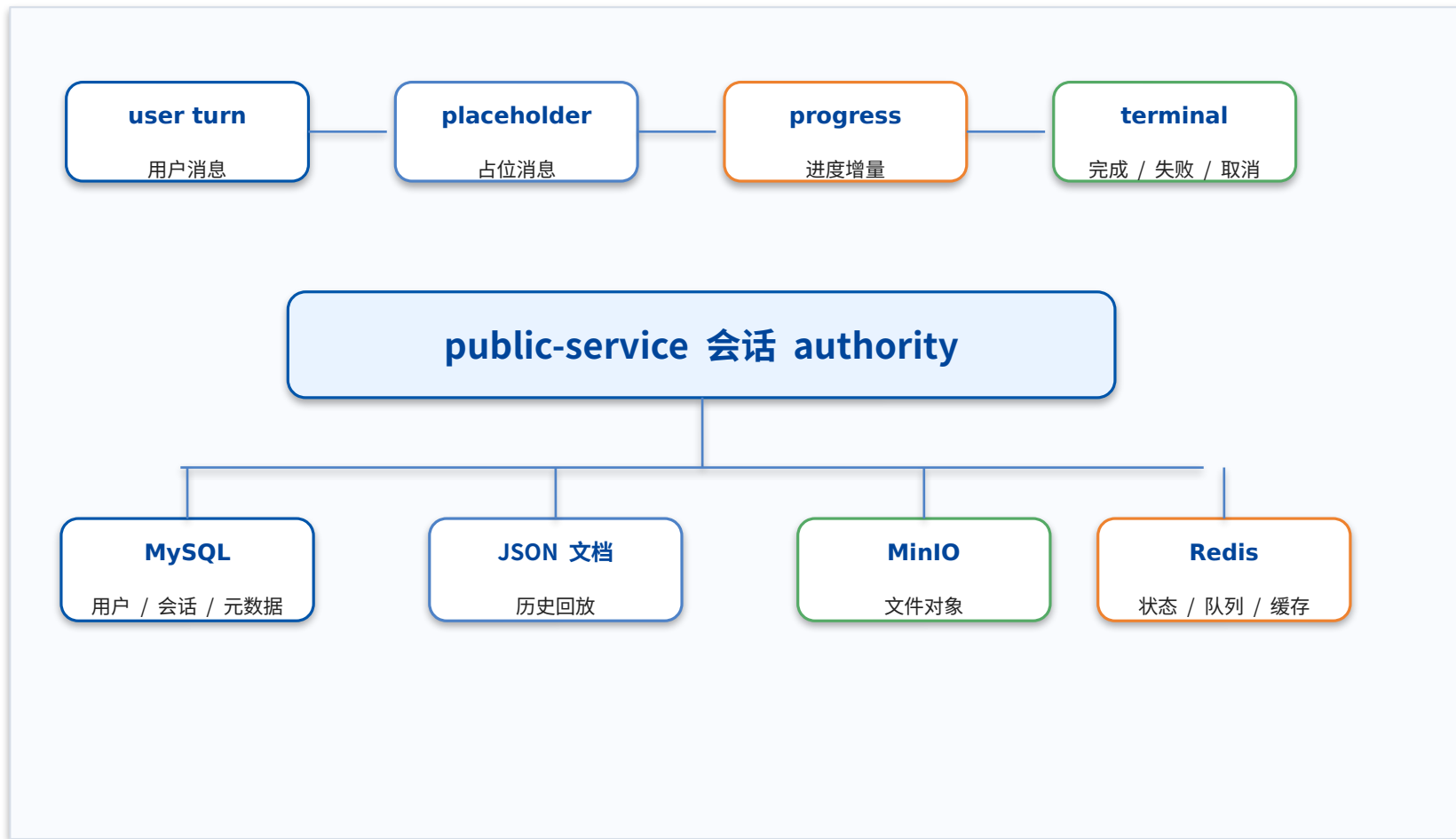
Redis 保存 last_seq 与任务状态，
刷新后可补齐遗漏事件。
前端可根据最新序号继续接收后续事件。

已实现：终态落库

完成、失败、取消统一写入 public-service ,
会话历史保持明确终态。
刷新后展示结果与后端状态一致。

实现消息落库与多存储协同

用户消息、占位消息、过程事件和终态消息形成可恢复的会话记录。



已实现：会话权威

public-service 统一写入用户消息、assistant 占位消息、进度增量和终态消息。前端历史回放以公共服务记录为准。

已实现：存储落点

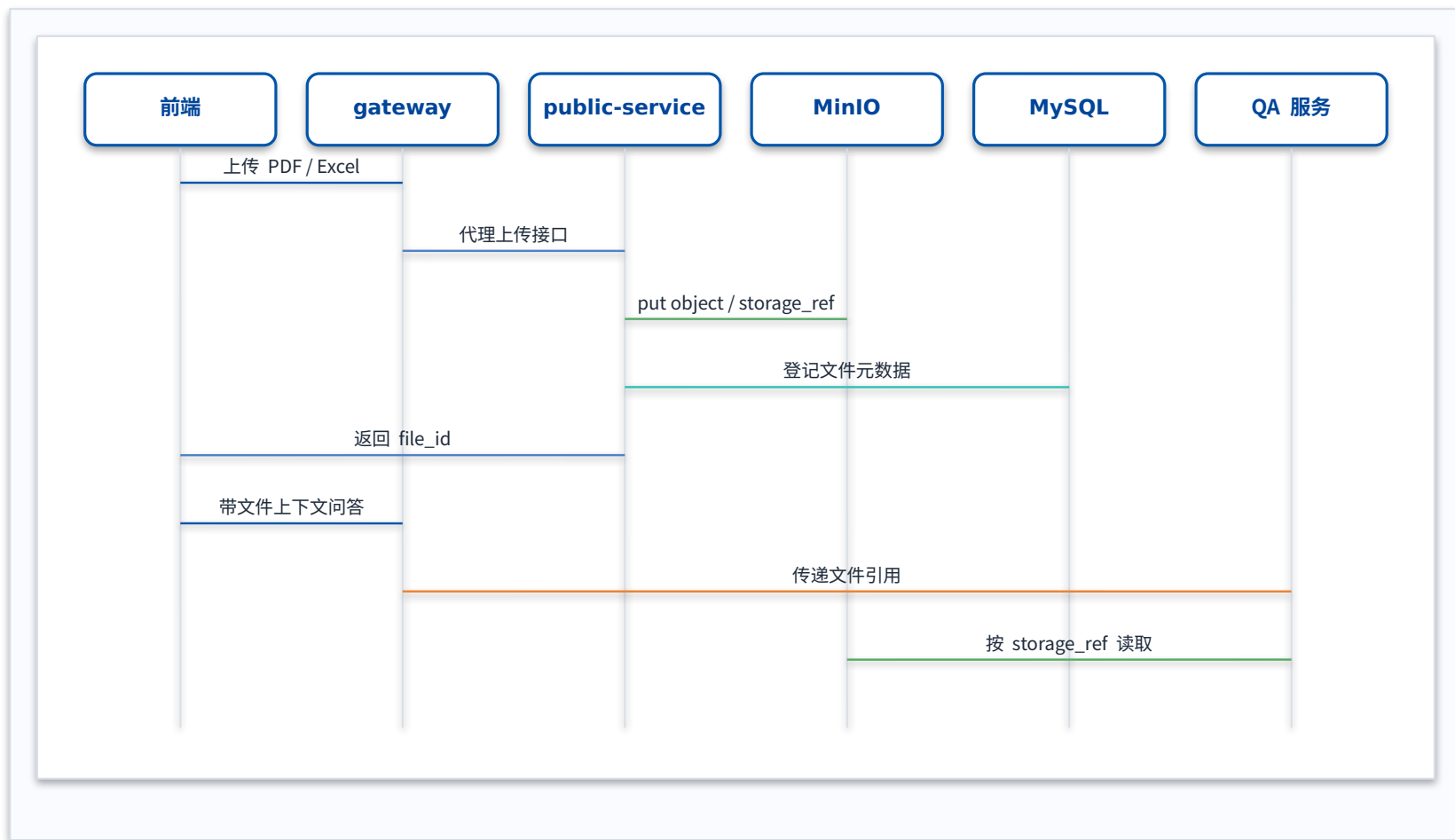
MySQL 保存用户、会话和元数据，JSON 记录支撑历史回放，MinIO 保存文件与原文对象。业务记录和对象引用可以相互对应。

已实现：运行状态

Redis 承接任务状态、队列和短期缓存，用于刷新恢复、并行任务和事件回放。任务排队、状态维护、消息终态统一协同。

实现文件上传与 MinIO 持久化

文件对象写入 MinIO，业务侧保留 file_id 与 storage_ref 供后续链路复用。



已实现：对象写入

上传文件写入 MinIO，
并在 MySQL 登记 file_id 与 storage_ref。
文件对象可在后续会话和任务中复用。

已实现：引用读取

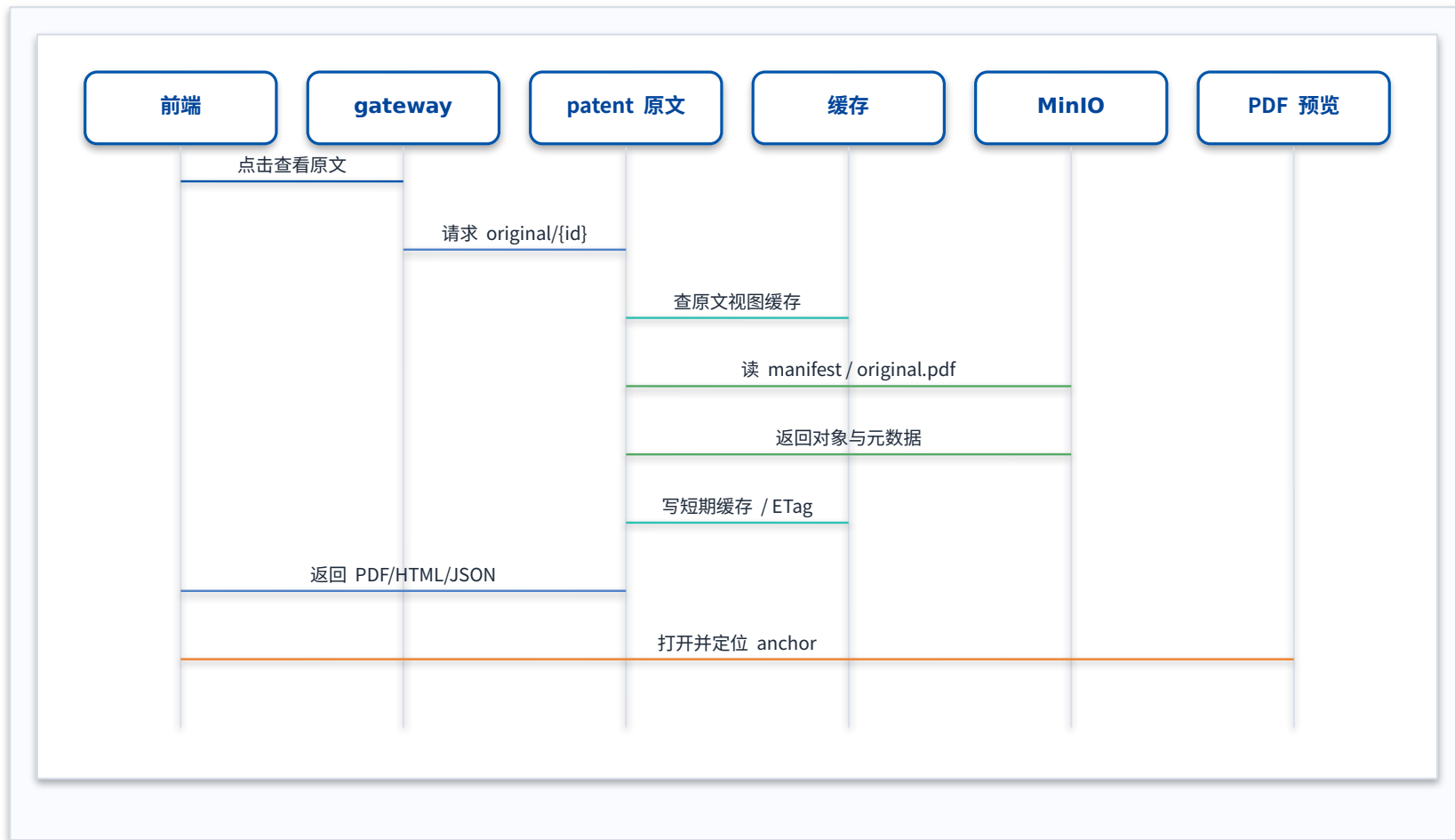
问答链路传递文件引用，
执行服务按 storage_ref 读取对象。
文件上下文通过稳定引用进入执行链路。

已实现：元数据闭环

前端获得 file_id 后参与会话与问答，
文件对象与业务记录可以对应。
后续预览、追踪和清理有依据。

实现点击查看原文与 MinIO 读取

答案中的原文引用可从 MinIO 读取专利原文对象，并支持打开与定位。



已实现：原文读取

点击查看原文后，服务按 original id 读取 manifest 与原文 PDF。原文对象来源统一落到 MinIO。

已实现：预览定位

返回 PDF / HTML / JSON 等信息，前端打开预览并定位 anchor。答案引用可以跳转到对应原文位置。

已实现：缓存协同

短期缓存与 ETag 支持连续查看，MinIO 作为原文对象来源。重复打开原文时减少链路重复读取。

实现前端配套页面与交互

围绕后端实现补齐页面入口，覆盖管理、任务恢复、文件上传和原文查看。

已实现：问题列表

右侧问题列表
点击定位到对应回答区域
长会话回看更直接

已实现：管理员页面

用户 / 部门 / 人员
绑定关系管理
批量导入与状态维护

已实现：注册与个人中心

安全问题
人员绑定
个人信息补全

已实现：文件与原文查看

文件上传与预览
答案引用查看原文
MinIO 链路前端入口

实现后：前端页面与后端接口同步落地，管理、任务恢复、文件与原文查看形成完整链路。

已实现：问题定位

右侧问题列表点击后定位到对应回答区域，
便于回看长会话。
长任务刷新恢复后仍可快速定位上下文。

已实现：管理页面

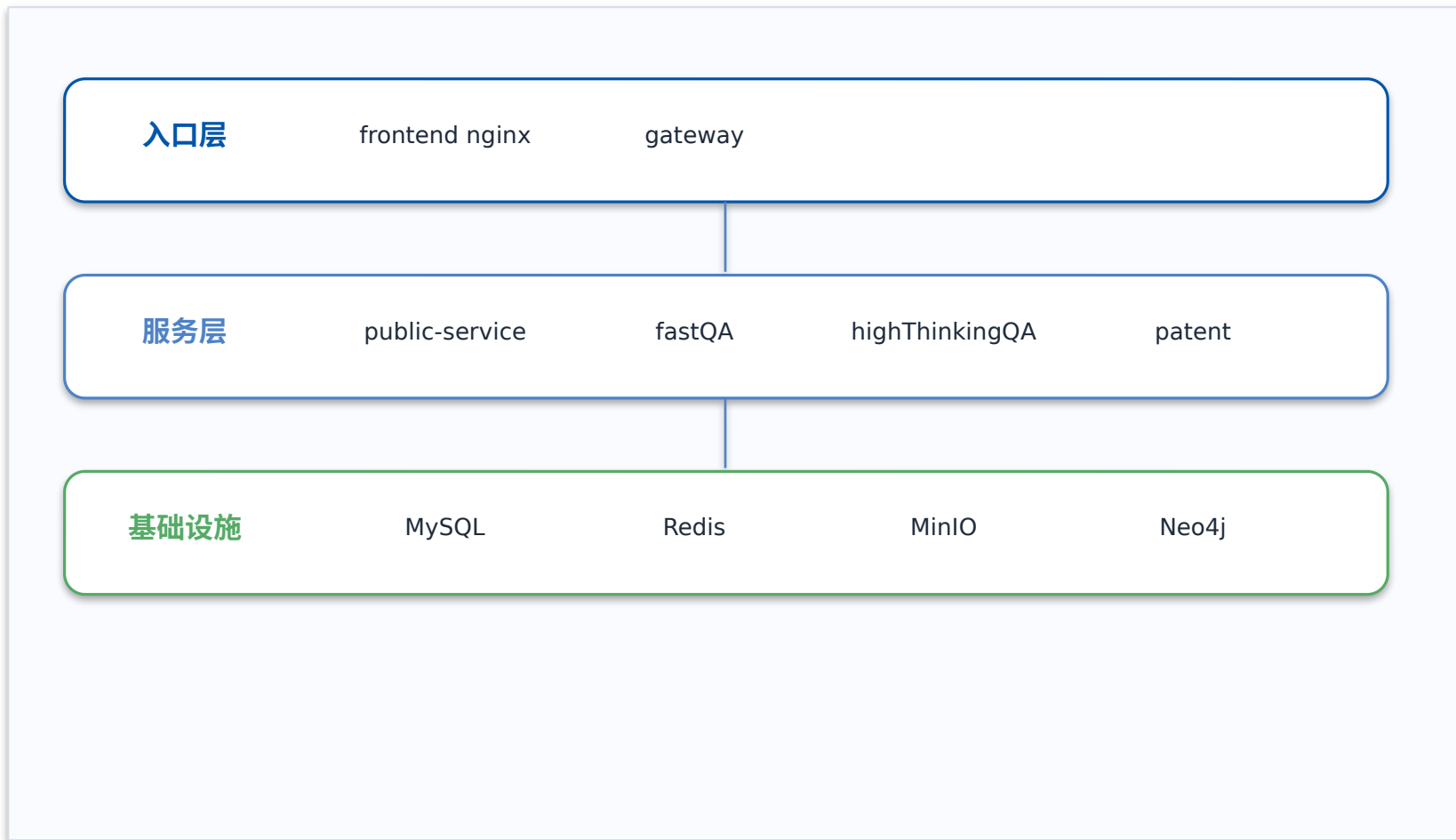
管理员页面、注册页和个人中心
承接组织人员与账号绑定接口。
后台实现有对应的前端操作面。

已实现：文件原文

上传、预览和查看原文的操作入口
与后端 MinIO 链路保持一致。
用户从答案到原文查看形成闭环。

实现部署编排与运行检查

部署部分简要说明多服务编排、统一入口和基础设施协同。



已实现：Compose 编排

使用 Docker Compose 编排 gateway、public-service、QA 服务和基础设施。交付时可按服务层次启动与检查。

已实现：统一入口

前端 nginx 与 gateway 对外承接访问，后端服务在内部按路由协作。外部访问路径保持集中和稳定。

已实现：运行检查

保留多服务启动、状态检查与基础设施连通性校验脚本。部署后可快速确认核心服务可用。

已实现内容汇总

核心服务链路、管理后台和任务模型已形成完整闭环。

服务结构已实现

gateway 统一入口与路由
public-service 公共接口
fast / thinking / patent 独立执行
基础设施协同落点

管理后台已实现

用户 / 部门 / 人员维护
账号人员绑定与状态控制
管理员修复和批量导入
用户侧信息补全

任务模型已实现

task_id 状态生命周期
刷新恢复与重新附着
多任务并行和真实取消
事件回放与终态落库

存储链路已实现

MySQL 会话和元数据
JSON 历史回放记录
MinIO 文件与原文对象
Redis 状态、队列、缓存

当前已实现从统一入口、公共服务、任务模型到文件与原文存储的核心链路。

已实现：架构闭环

frontend-vue、gateway、public-service
与 QA 服务边界已形成闭环。
入口、公共服务、执行服务职责清晰。

已实现：业务闭环

管理员、组织人员、任务模型、
消息持久化与文件原文链路已贯通。
核心流程可以围绕真实会话连续运行。

已实现：交付闭环

部署编排、健康检查
与基础设施协同已纳入交付资料。
后续继续补充测试和运维观测。

后续完善计划

围绕测试、观测、存储一致性和部署文档继续完善工程化交付。

测试覆盖

补齐登录鉴权、任务恢复、取消、终态消息、MinIO 原文读取等关键链路测试

可观测性

完善任务状态、消息落库、对象读取、服务健康等日志与指标观测

存储一致性

加强 MinIO 对象与 MySQL 元数据之间的一致性校验和异常处理

部署 runbook

整理启动、回滚、配置、排障和健康检查说明，形成可复用交付文档

自动化测试

围绕登录鉴权、任务状态、消息终态和 MinIO 链路补齐测试。覆盖刷新恢复、取消和原文读取等关键路径。

运维可观测性

补充任务、消息、存储和服务健康的日志与指标观测。便于定位长任务执行和对象读取状态。

部署文档整理

整理启动、回滚、排障和配置说明，形成交付 runbook。让多服务部署过程可复用、可检查。